

Developing Bubble Shooter Game Using Graphics and Visual Computing

A Project Report

Submitted by

Group-20

Rishabh Gupta (IIT2023025)

Aaryan Kuntal (IIT2023014)

Kumar Prateek (IIT2023087)



Department of Information Technology
Indian Institute of Information Technology, Allahabad

April 2025

Contents

1	Abstract	2
2	Introduction	2
2.1	Core Features	2
3	Methodology	3
3.1	Game Initialization	3
3.2	Bubble Mechanics	3
3.3	Collision Detection and DFS	4
3.4	Game Loop and User Input	4
4	Planning and Implementation	4
4.1	Game Architecture	4
4.2	Core Algorithms	5
4.3	Rendering and Graphics	5
4.4	Game Loop	6
5	Workflow Diagram	7
6	Conclusion	7
7	References	8

1 Abstract

This paper presents the development of a 2D Bubble Shooter game using OpenGL and the GLUT library. The game challenges players to shoot bubbles from the bottom of the screen, aiming to match three or more bubbles of the same color to pop them, with the goal of preventing the bubbles from reaching the bottom. The implementation involves capturing user input for aiming and shooting, employing collision detection to place bubbles, and utilizing depth-first search (DFS) to identify and remove clusters of matching bubbles. The game architecture includes a game loop that continuously updates the game state, handles user input, and renders graphics. The results demonstrate smooth gameplay, accurate collision detection, and an engaging difficulty progression system. The game successfully meets its objectives, providing a visually appealing and functional gaming experience with a dynamic scoring system and increasing levels of difficulty as the player progresses.

2 Introduction

The Bubble Shooter Game is a popular puzzle and arcade-style game that challenges the player's aim and strategy. The game is set in a two-dimensional environment where the player launches bubbles from the bottom of the screen towards a cluster of bubbles located at the top. The primary objective is to prevent the bubbles from reaching the bottom by strategically aiming and shooting bubbles of varying colors to create combinations. When three or more bubbles of the same color come into contact, they burst, earning the player points and clearing space on the screen.

The game requires the player to calculate angles and trajectory to bounce bubbles off the walls and hit desired targets. The speed and complexity increase as the game progresses, requiring faster decision-making and precision.

2.1 Core Features

- **Game Objective:** The player's goal is to pop all the bubbles by forming groups of three or more adjacent bubbles of the same color, preventing the cluster from reaching the bottom of the screen.

- **Shooting Mechanism:** The player controls a shooter at the bottom, adjusting the direction of the shot using the mouse or keyboard. The shooter fires bubbles at a specific angle to strategically pop bubbles in the upper cluster.
- **Game Progression:** As the player successfully bursts bubbles, the cluster rearranges itself, and new rows of bubbles may drop down, increasing the difficulty. The game continues until all bubbles are cleared or the player loses.
- **Game Over:** The game ends either when the bubbles reach the bottom of the screen, signifying defeat, or when the player successfully eliminates all bubbles on the screen, winning the game.

3 Methodology

The Bubble Shooter Game is implemented using the OpenGL Utility Toolkit (GLUT) for rendering graphics and handling user input. The methodology follows a structured approach, incorporating key components like bubble mechanics, collision detection, and player input to create a functional and engaging gameplay experience.

3.1 Game Initialization

The game begins by initializing the GLUT window and setting up the coordinate system with `gluOrtho2D` to create a 2D plane. The game window is set at 480x640 pixels, with a starting state for the player-controlled shooter and an initial cluster of bubbles at the top. The `inits()` function is responsible for initializing the necessary variables, including bubble positions and colors.

3.2 Bubble Mechanics

Bubbles are represented in a grid-like structure. The core mechanic revolves around the player aiming and shooting bubbles from the bottom of the screen using the mouse input, which is handled by the `mymouse()` function. The bubble trajectory is determined by an angle (`theta`), and when a bubble reaches the top or collides with other bubbles, it snaps into the grid and checks for matches.

3.3 Collision Detection and DFS

The game uses depth-first search (DFS) to detect adjacent bubbles of the same color. When a new bubble is added to the grid, the `dfs()` function recursively checks neighboring bubbles for matches. If a match of three or more bubbles of the same color is found, those bubbles are popped, and the game grid is updated accordingly.

3.4 Game Loop and User Input

The game operates in a continuous loop using `glutMainLoop()`, which constantly refreshes the display via the `display()` function. User input is handled via mouse events, allowing the player to aim and shoot bubbles dynamically. Game progression is monitored to check for victory or game over conditions when bubbles reach the bottom of the screen.

4 Planning and Implementation

The successful development of the Bubble Shooter game involves a well-structured approach that integrates various programming methodologies, algorithms, and data structures. The following outlines the core components of our planning and implementation strategy:

4.1 Game Architecture

The architecture of the Bubble Shooter game follows a procedural design pattern, focusing on modularity and simplicity. The game consists of the following components:

- **Game Engine:** Responsible for handling the overall game loop, including initialization, rendering, updating game states, and responding to user inputs like mouse clicks for aiming and shooting.
- **Game Objects:** Each bubble is represented as a structure with properties such as color, position, and state (active or inactive). Additional game objects include the shooter, game grid, score display, and game-over screen.

- **Collision Detection:** The game uses bounding box collision detection to determine when bubbles come into contact, enabling the identification and removal of bubble clusters.

4.2 Core Algorithms

Several algorithms were implemented to manage the core functionality of the game:

1. Bubble Shooting Algorithm:

- **Input Handling:** User input for aiming and shooting bubbles is captured. Trigonometric calculations compute the shooting angle and trajectory based on the shooter's position.
- **Bubble Launch:** The bubble's position is updated as it travels along its trajectory. The algorithm checks for collisions with boundaries or other bubbles to snap the bubble to the grid.

2. Collision Detection:

- **Bounding Box Collision:** Axis-aligned bounding box (AABB) collision detection determines if a shot bubble collides with existing bubbles.
- **Cluster Detection:** Once a bubble is placed on the grid, a depth-first search (DFS) detects clusters of three or more bubbles of the same color, removing matching bubbles.

3. Score Calculation:

A scoring system increments the player's score based on the number of bubbles popped. Larger clusters yield higher scores, encouraging strategic play.

4. Level Progression:

The game includes multiple levels, with increasing difficulty by adding more colors and speeding up the rate at which new rows of bubbles appear.

4.3 Rendering and Graphics

The OpenGL library handles graphical rendering:

- **Drawing Bubbles:** Bubbles are drawn as circles using trigonometric functions to define their shape and position on the grid.
- **Background and UI Elements:** Background, score display, and game-over messages are rendered using OpenGL for smooth visuals.

4.4 Game Loop

The game loop is structured as follows:

1. **Initialization:** The game initializes graphics and variables.
2. **Input Handling:** The game captures mouse input for aiming and shooting bubbles.
3. **Game State Update:** After each bubble is shot, the game checks for collisions and cluster matches.
4. **Rendering:** The screen is cleared and redrawn with updated game elements.
5. **Game Over Check:** The game verifies if bubbles have reached the bottom, determining a game-over state.

5 Workflow Diagram

```
"[Start Game]
|
v
[Main Menu] --> Options:
  1. About the Project
  2. Rules of the Game
  3. Game Mode or Restart
  4. Quit
|
+--> [View About Project] --> Return to Main Menu
|
+--> [View Rules] --> Return to Main Menu
|
+--> [Quit Game] --> [End]
|
v
[Select Game Mode/Restart Game]
|
v
[Game Screen]
|
+--> [User Input] (Mouse Clicks/Keypresses) --> Move player or interact with game elements
|
+--> [Game Logic] --> Process dice roll, player turns, and validate movements
|
v
[Game Updates]
|
+--> [Update Score/Status] --> Update player positions and display current game status
|
v
[Is Game Over?]
|
+--> [Yes] --> Show Result Screen --> Back to Main Menu or Quit
|
+--> [No] --> Continue Game
|
v
[End Game] --> Return to Main Menu or Quit"
```

6 Conclusion

The Bubble Shooter Game was successfully implemented using the OpenGL and GLUT libraries, combining fundamental concepts from graphics programming, data structures, and algorithmic thinking. The final product provides a smooth and engaging gameplay experience with progressively challenging levels, accurate collision detection, and dynamic scorekeeping. This project has not only demonstrated the potential of graphics libraries in creating interactive applications but also reinforced

core programming skills such as recursion (DFS), trigonometry, and real-time user input handling.

7 References

- [1] Donald Hearn Pauline Baker: Computer Graphics with OpenGL Version, 4 Edition, Pearson Education, 2011.
- [2] Edward Angel: Interactive Computer Graphics- A Top Down approach with th OpenGL, 5 edition. Pearson Education, 2008.
- [3] Jackie.L.Neider,Mark Warhol,Tom.R.Davis,"OpenGL Red Book",Second Revised Edition,2005. [4] www.opengl.org
- [4] <https://learnopengl.com/>